# 1. PROJECT OVERVIEW

### Project/ App Name

Creators Studio

---

### Project Description

This project is a mobile-responsive expense management application developed for a small fabric business owner. The application is intended to help the user record, manage, and track expenses related to purchasing fabrics and accessories used in garment production.

The application focuses mainly on maintaining purchasing records in a simple, fast, and organized manner.

The current version of the application includes:

- User authentication
- Fabric purchasing bill entry
- Accessory purchasing bill entry
- Dynamic form handling
- Automatic total price calculation

The application is planned to be scalable so that future modules such as payment bills, expense history tables, reports, and analytics can be added later without major structural changes.

---

## Main Objective

The main objective of this application is to replace manual expense tracking with a structured digital system that allows the business owner to:

- Enter purchasing expenses quickly
- Organize fabric and accessory purchase data
- Reduce manual calculation work
- Maintain records digitally
- Prepare the application for future reporting and analytics features

---

## Target User

The application is designed for:

- A single business owner
- Internal business usage only
- No multi-user management currently required

---

# 2. Core Features Implemented in Current Design

## 1. User Authentication

The application starts with a login page.

### Login Flow

- Existing users can log in using username and password.
- New users can register using:
    - Username
    - Password
    - Confirm Password

After successful login, the user is redirected to the dashboard.

---

## 2. Dashboard

The dashboard currently contains two main modules:

- Purchasing Bills
- Payment Bills

Currently, only the Purchasing Bills module is fully designed and implemented in the requirement phase.

The Payment Bills module is reserved for future development.

---

# Purchasing Bills Module

Inside the Purchasing Bills module, the user must select one of the following categories:

- Fabrics
- Accessories

---

# Fabric Purchasing Module

The Fabric Purchasing module is used to store expenses related to fabric purchases.

The user must enter:

- Fabric Name
- Fabric Type
- Colour
- GSM
- Weight
- Rib
- Price Per Kg

---

## Dynamic Dropdown Behaviour

### Fabric Name

The Fabric Name field is a dropdown selection field.

If the user selects:

- “Others”

then the application must allow manual text input for entering a custom fabric name.

---

### Fabric Type

The Fabric Type field also behaves similarly:

- Dropdown selection
- “Others” option enables manual entry

---

## Dynamic Multiple Entry Feature

The Fabric module supports adding multiple colour entries for the same fabric purchase.

An Add button is provided that dynamically creates additional rows containing:

- Colour

- GSM
- Weight
- Rib
- Price Per Kg

This allows the user to enter multiple colour variations under the same fabric purchase entry.

---

## Automatic Calculation

The application must automatically calculate Total Price using:

- Weight
- Price Per Kg

The calculated total should update dynamically whenever values are changed.

---

## Fixed Units

The following units are fixed and cannot be modified by the user:

- Weight → KG
- Rib → g
- Price → ₹ symbol

These units are displayed automatically in the UI.

---

## Submission Flow

After entering all required data:

- User clicks Submit
- Data is stored in the database
- User is redirected back to Dashboard

---

# Accessories Purchasing Module

The Accessories module is used to store accessory purchase expenses.

The user first selects an Accessory Name from a dropdown.

Depending on the selected accessory type, different input fields are displayed dynamically.

---

# Cone Accessory

If the user selects:

- Cone

the following fields must be displayed:

- Colour Name
- Colour Code
- Unit
- Price Per Unit

---

## Multiple Dynamic Entries

The Cone module also supports dynamic multiple entries using an Add button.

The Add button creates new rows containing:

- Colour Name
- Colour Code
- Unit
- Price Per Unit

This allows multiple cone colour entries within the same purchase.

---

## Automatic Calculation

Total Price is automatically calculated using:

- Unit
- Price Per Unit

---

# Size Pattern Accessory

If the user selects:

- Size Pattern

the following fields must be displayed:

- Brand Name
- Style Number
- Style Name
- Price

---

# Others Accessory

If the user selects:

- Others

the following fields must be displayed:

- Name
- Unit / Kg / Gross
- Price

---

# Current Project Scope

The current project scope includes:

- Authentication system
- Dashboard navigation
- Fabric purchasing entry system
- Accessory purchasing entry system
- Dynamic forms
- Automatic price calculations
- Database storage

---

# Future Enhancements

The following modules are planned for future implementation:

- Payment Bills Module
- Expense Table View
- Search and Filter Features
- Reports Module

- PDF Export
- Expense Analytics Dashboard
- Monthly Expense Summary
- Edit/Delete Records
- Supplier Management
- Inventory Tracking

---

# Technical Architecture

The application will follow:

- REST API architecture
- Layered backend architecture
- MVC design pattern

---

# 3. Planned Technology Stack

## Frontend

- HTML
- CSS
- JavaScript

---

## Backend

- Core Java
- JDBC
- Spring Framework
- Spring Boot
- Spring MVC
- Spring Data JPA

---

## Database

- MySQL

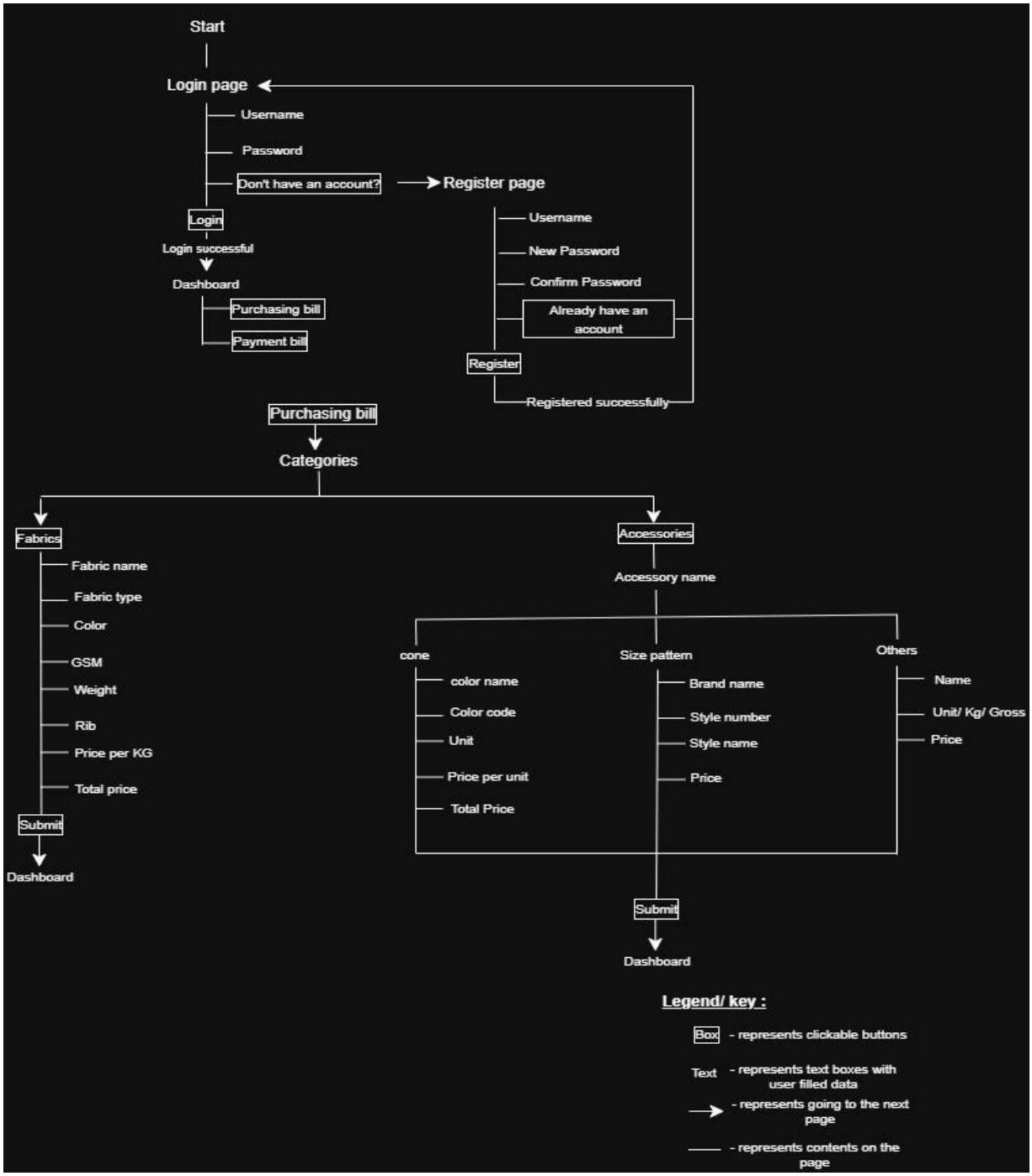
---

# Development Goal

The goal of this project is to create a scalable, maintainable, and user-friendly expense management application that simplifies purchase recording operations for a fabric business while also serving as a strong full-stack development project using Java and Spring Boot technologies.

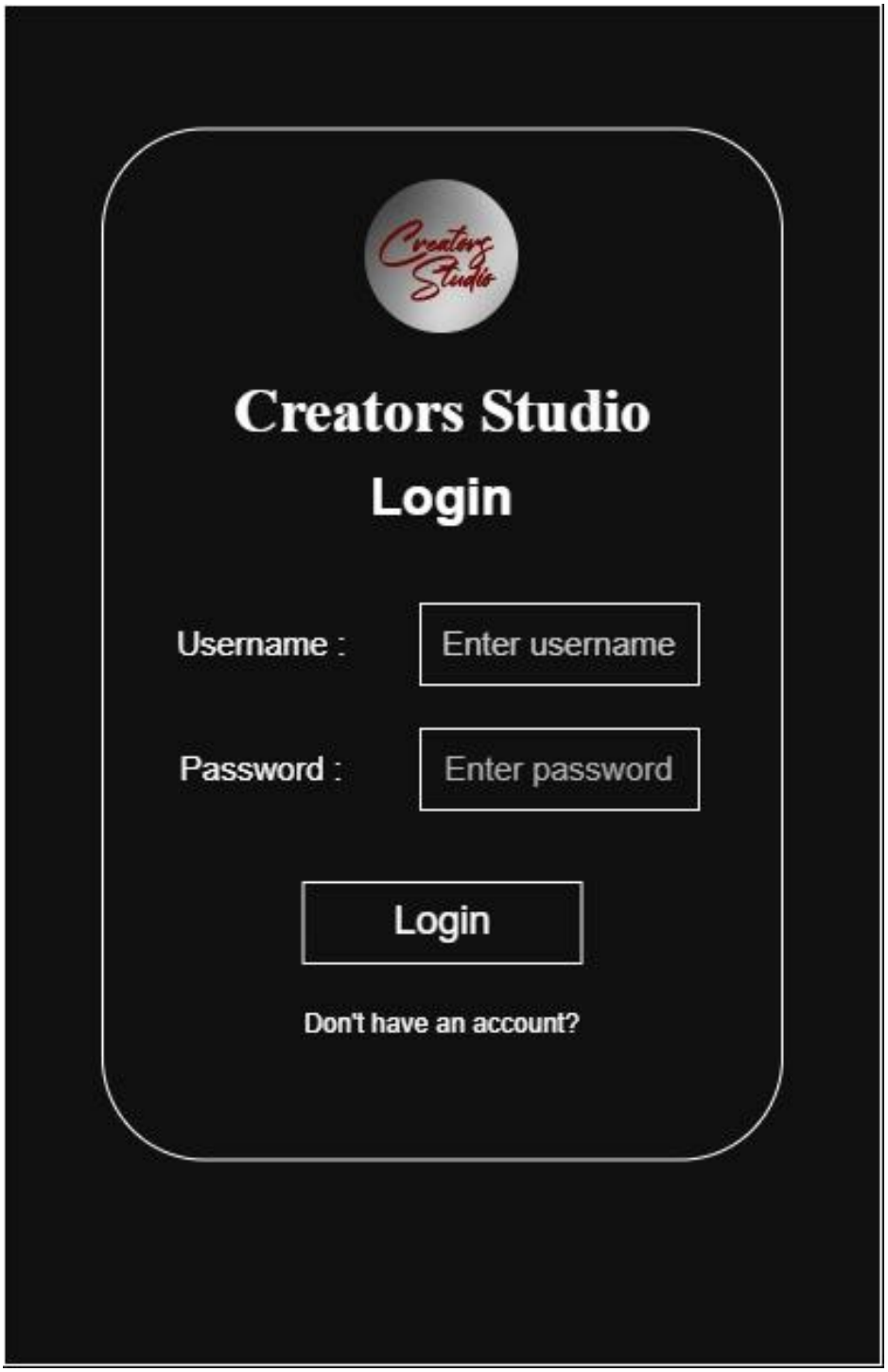
---

# 4. Screen flow



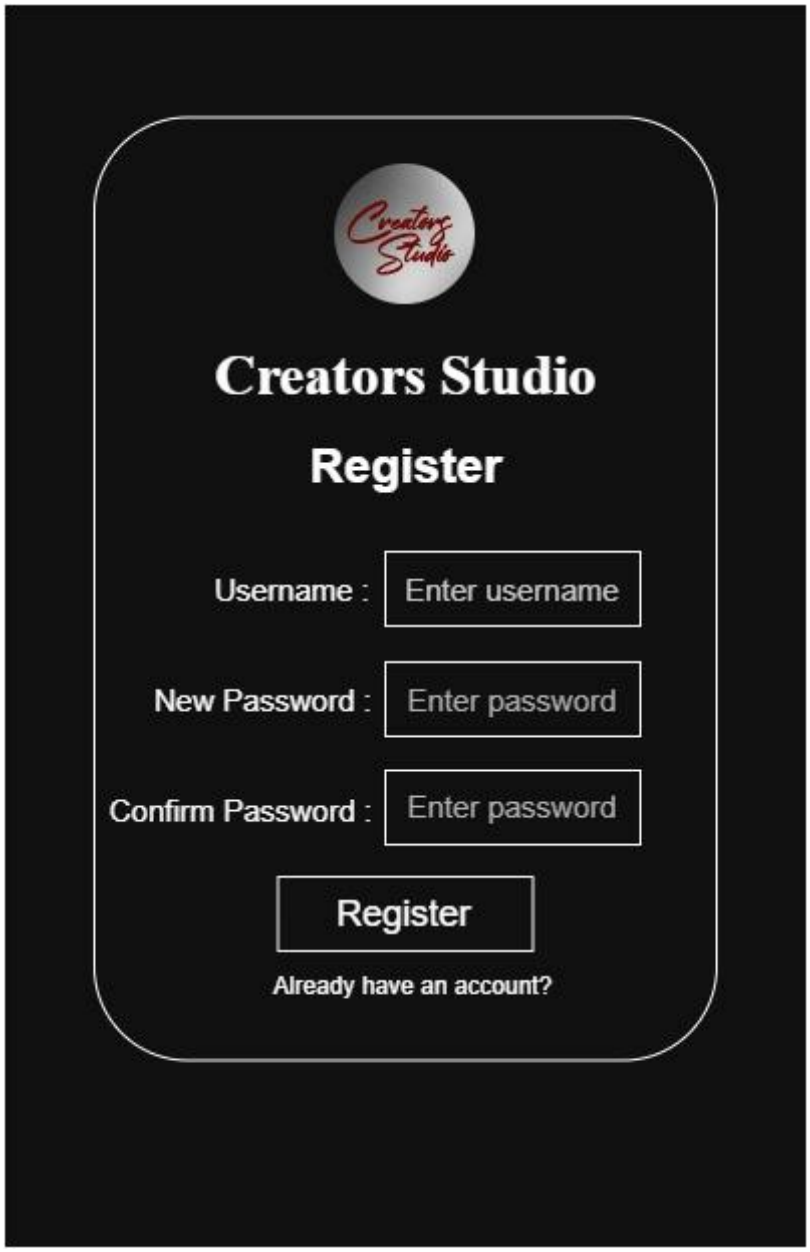
# 5. UI Wireframes

## i. Login Page



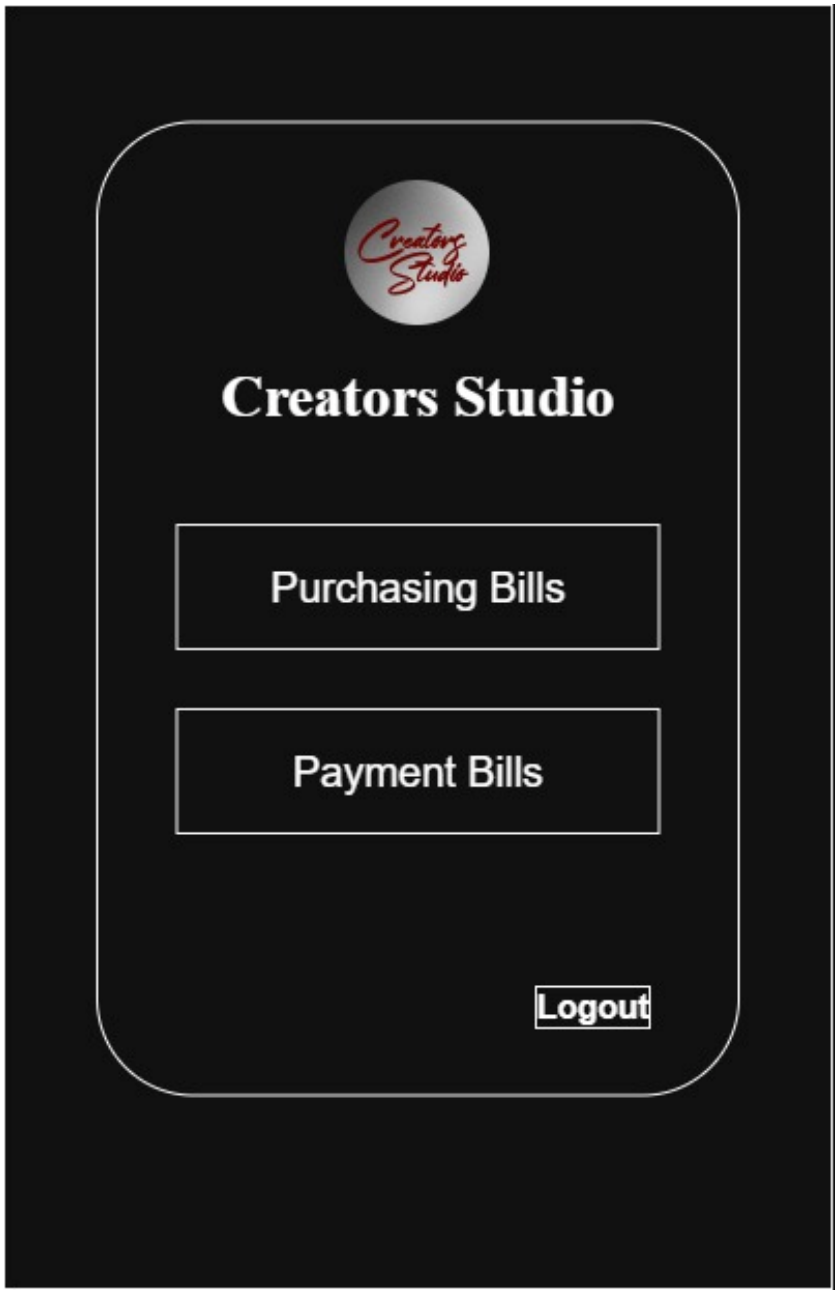

This page is for login-in into the app.

## ii. Register page




This page is for registering into app.

## iii. Dashboard

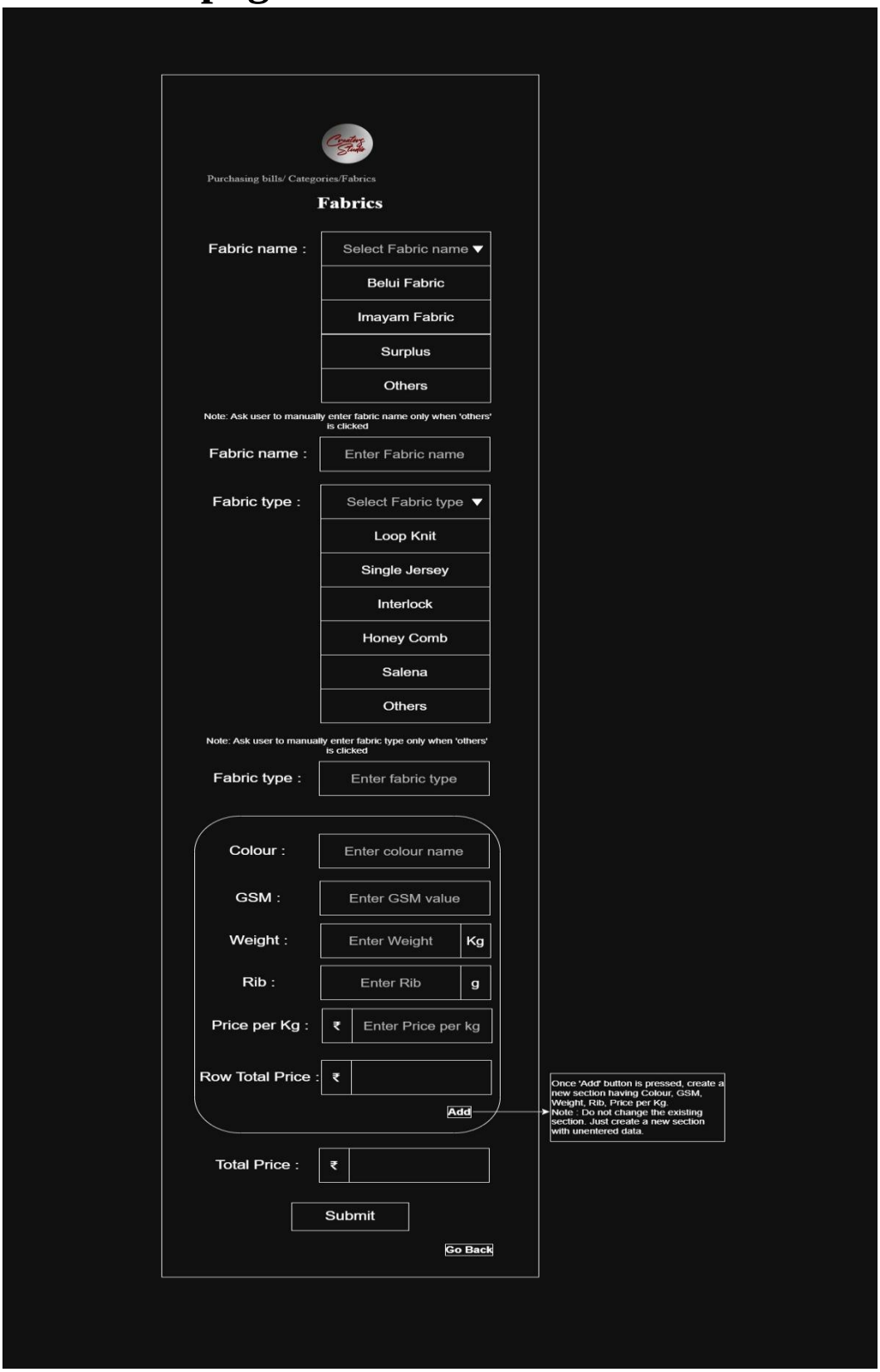



This page is the dashboard page of the app.

## iv. Purchasing bills page

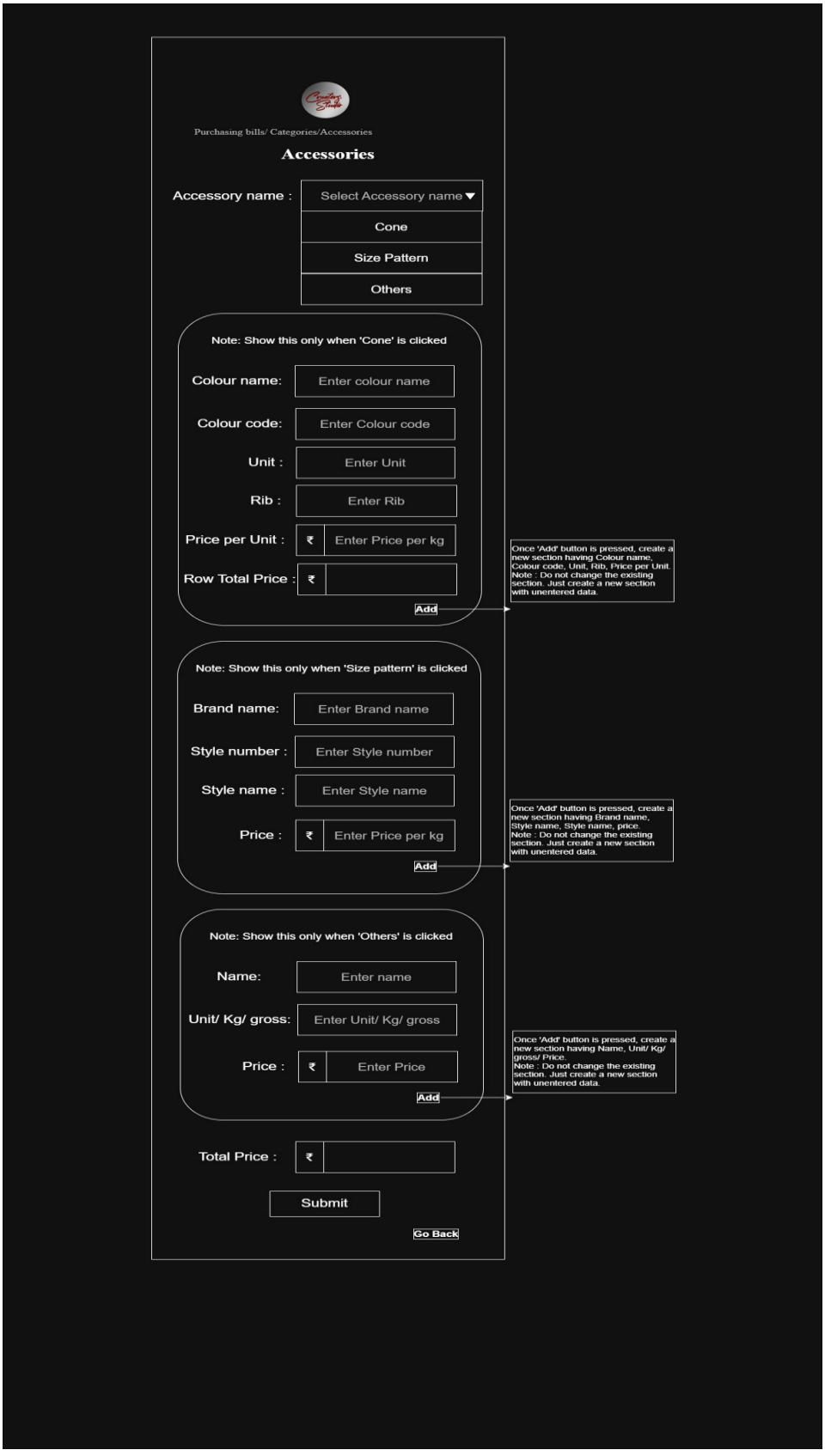

This page is used to know whether the user purchased fabric or accessory.

## v. Fabrics page




This page allows the user to get fabric details and add multiple fabric colour entries dynamically.

# vi. Accessories page



This page allows the user to add accessory details and add multiple cone accessory colour and other entries dynamically.

# 6. Database Blueprint

**Table 1 - user:**

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| Id | bigint | Primary key, AUTO_INCREMENT | Unique user ID |
| Username | varchar(50) | Unique, NOT NULL | Unique username |
| Password | varchar(255) | NOT NULL | Encrypted password |

**Table 2 – Fabric**

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| id | bigint | Primary key, AUTO_INCREMENT | Unique fabric ID |
| Fabric_name | Varchar(100) | NOT NULL | |
| Fabric_type | Varchar(100) | NOT NULL | |
| Total_price | decimal | NOT NULL | |
| Purchase_date | Datetime | NOT NULL | Records the purchased date |
| Created_at | timestamp | NOT NULL | Records the purchased time |

**Table 3 – Fabric_items**

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| id | bigint | Primary key, AUTO_INCREMENT | Unique fabric ID |
| Fabric_id | bigint | Foreign key | |
| colour | Varchar(100) | NOT NULL | |
| GSM | int | NOT NULL | |
| Weight | decimal | NOT NULL | |
| Rib | decimal | NOT NULL | |
| Price_per_kg | decimal | NOT NULL | |
| Row_total | decimal | NOT NULL | |

## Table 4 – Accessory

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| id | bigint | Primary key, AUTO_INCREMENT | Unique accessory ID |
| Accessory_name | Varchar(100) | NOT NULL | |
| Total_price | decimal | NOT NULL | Combined final total |

## Table 5 – Cone

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| id | bigint | Primary key, AUTO_INCREMENT | Unique cone ID |
| Accessory_id | bigint | Foreign key -> accessory.id | Linked accessory |
| Purchase_date | Datetime | NOT NULL | Records the purchased date |
| Created_at | timestamp | NOT NULL | Records the purchased time |

## Table 6 – Cone_items

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| id | bigint | Primary key, AUTO_INCREMENT | Unique cone ID |
| Cone_id | bigint | Foreign key | |
| colour_name | Varchar(100) | NOT NULL | |
| Colour_code | Varchar(100) | NOT NULL | |
| unit | decimal | NOT NULL | |
| Price_per_unit | decimal | NOT NULL | |
| Row_total | decimal | NOT NULL | Row subtotal |

## Table 7 – Size_pattern

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| Id | bigint | Primary key, AUTO_INCREMENT | Unique size pattern ID |
| Accessory_id | bigint | Foreign key -> accessory.id | Linked accessory |
| brand_name | Varchar(100) | NOT NULL | |
| style_number | Int | NOT NULL | |
| Style_name | Varchar(100) | NOT NULL | |
| price | decimal | NOT NULL | |

**Table 8 – others**

| Column | Data type | Constraints | Purpose |
|---|---|---|---|
| id | bigint | Primary key, AUTO_INCREMENT | Unique cone ID |
| Accessory_id | bigint | Foreign key -> accessory.id | Linked accessory |
| Items_name | Varchar(100) | NOT NULL | |
| unit | decimal | NOT NULL | |
| price | decimal | NOT NULL | |

# 7. API BLUEPRINT

## Base URL

`/api`

---

# AUTH MODULE APIs

## Register User

`POST /api/auth/register`

### Purpose

Create new account

---

## Login User

`POST /api/auth/login`

### Purpose

Login existing user

---

# DASHBOARD MODULE APIs

## Get Dashboard Data

`GET /api/dashboard`

### Purpose

Load dashboard modules

---

# FABRIC PURCHASE MODULE APIs

## Create Fabric Purchase

`POST /api/purchases/fabrics`

### Purpose

Save new fabric purchasing bill

---

## Get All Fabric Purchases

`GET /api/purchases/fabrics`

### Purpose

Get all fabric purchase records

---

## Get Fabric Purchase By ID

`GET /api/purchases/fabrics/{id}`

### Purpose

Get single fabric purchase details

---

## Update Fabric Purchase

`PUT /api/purchases/fabrics/{id}`

### Purpose

Update fabric purchase details

---

## Delete Fabric Purchase

`DELETE /api/purchases/fabrics/{id}`

### Purpose

Delete fabric purchase entry

---

# ACCESSORY PURCHASE MODULE APIs

## Create Accessory Purchase

`POST /api/purchases/accessories`

### Purpose

Save new accessory purchasing bill

---

## Get All Accessory Purchases

`GET /api/purchases/accessories`

### Purpose

Get all accessory purchase records

---

## Get Accessory Purchase By ID

`GET /api/purchases/accessories/{id}`

### Purpose

Get single accessory purchase details

---

## Update Accessory Purchase

`PUT /api/purchases/accessories/{id}`

### Purpose

Update accessory purchase details

---

## Delete Accessory Purchase

`DELETE /api/purchases/accessories/{id}`

### Purpose

Delete accessory purchase entry

---

# CONE ACCESSORY APIs

## Create Cone Entry

`POST /api/accessories/cone`

### Purpose

Save cone accessory details

---

## Get All Cone Entries

`GET /api/accessories/cone`

### Purpose

Get all cone entries

---

## Get Cone Entry By ID

`GET /api/accessories/cone/{id}`

### Purpose

Get single cone entry

---

## Update Cone Entry

`PUT /api/accessories/cone/{id}`

### Purpose

Update cone details

---

## Delete Cone Entry

`DELETE /api/accessories/cone/{id}`

### Purpose

Delete cone entry

---

# SIZE PATTERN APIs

## Create Size Pattern Entry

`POST /api/accessories/size-pattern`

### Purpose

Save size pattern details

---

## Get All Size Pattern Entries

`GET /api/accessories/size-pattern`

### Purpose

Get all size pattern entries

---

## Get Size Pattern Entry By ID

`GET /api/accessories/size-pattern/{id}`

### Purpose

Get single size pattern entry

---

## Update Size Pattern Entry

`PUT /api/accessories/size-pattern/{id}`

### Purpose

Update size pattern details

---

## Delete Size Pattern Entry

`DELETE /api/accessories/size-pattern/{id}`

### Purpose

Delete size pattern entry

---

# OTHER ACCESSORY APIs

## Create Other Accessory Entry

`POST /api/accessories/others`

### Purpose

Save other accessory details

---

## Get All Other Accessory Entries

`GET /api/accessories/others`

### Purpose

Get all other accessory entries

---

## Get Other Accessory Entry By ID

```
GET /api/accessories/others/{id}
```

### Purpose

Get single other accessory entry

---

## Update Other Accessory Entry

```
PUT /api/accessories/others/{id}
```

### Purpose

Update other accessory details

---

## Delete Other Accessory Entry

```
DELETE /api/accessories/others/{id}
```

### Purpose

Delete other accessory entry

---

# RECOMMENDED CONTROLLER STRUCTURE

```
AuthController
DashboardController
FabricPurchaseController
AccessoryPurchaseController
ConeController
SizePatternController
OthersController
```

---

# RECOMMENDED SERVICE STRUCTURE

```
AuthService
DashboardService
FabricPurchaseService
AccessoryPurchaseService
ConeService
SizePatternService
OthersService
```

---

# REQUEST METHODS USED

```
GET     -> Fetch data
POST    -> Create data
PUT     -> Update data
DELETE  -> Delete data
```

---

# FUTURE MODULE PLACEHOLDER

```
Payment Bills Module
Expense Table Module
Reports Module
Search Module
Filter Module
Export PDF Module
```

# 8. BUSINESS RULES

## Authentication Rules

### User Registration

- Each username must be unique.
- Username cannot be empty.
- Password cannot be empty.
- Confirm Password must exactly match Password.
- Password must be stored in encrypted format in the database.
- Duplicate usernames are not allowed.

---

### User Login

- Only registered users can log in.
- Username and password must match stored credentials.
- Invalid username or password must display an error message.
- After successful login, user must be redirected to Dashboard.

---

# Dashboard Rules

- Dashboard is displayed only after successful login.
- Dashboard currently contains:
  - Purchasing Bills
  - Payment Bills
- Payment Bills module is reserved for future implementation.
- User must be able to navigate to Purchasing Bills module from Dashboard.

---

# Purchasing Bills Rules

- User must select one category before entering purchase details.
- Available categories:
  - Fabrics
  - Accessories
- Data must not be saved until Submit button is clicked.

---

# Fabric Module Rules

## Fabric Name Rules

- Fabric Name must be selected from dropdown.
- Dropdown must contain predefined fabric names.
- Dropdown must include “Others” option.
- If “Others” is selected, user must be allowed to manually enter fabric name.
- Fabric Name field is mandatory.

---

## Fabric Type Rules

- Fabric Type must be selected from dropdown.
- Dropdown must contain predefined fabric types.
- Dropdown must include “Others” option.
- If “Others” is selected, user must be allowed to manually enter fabric type.

- Fabric Type field is mandatory.

---

## Colour Rules

- Colour field is manual entry.
- Colour field cannot be empty.

---

## GSM Rules

- GSM must be numeric.
- GSM cannot be negative.
- GSM field is mandatory.

---

## Weight Rules

- Weight must be numeric.
- Weight cannot be negative or zero.
- Weight unit must always remain “KG”.
- User must not be allowed to edit or remove “KG”.
- Weight field is mandatory.

---

## Rib Rules

- Rib must be numeric.
- Rib cannot be negative or zero.
- Rib unit must always remain “g”.
- User must not be allowed to edit or remove “g”.
- Rib field is mandatory.

---

## Price Per Kg Rules

- Price Per Kg must be numeric.
- Price cannot be negative or zero.
- Currency symbol ₹ must always be displayed.
- User must not be allowed to edit or remove currency symbol.
- Price Per Kg field is mandatory.

# Dynamic Row Rules — Fabric Module

- User can dynamically add multiple rows using Add button.
- Newly added rows must contain:
  - Colour
  - GSM
  - Weight
  - Rib
  - Price Per Kg
- Multiple rows belong to same fabric purchase entry.
- Each row must independently calculate its own Row Total using:

```
Row Total = Weight × Price Per Kg
```

- A Total Price must be calculated by adding all Row Totals together.
- Empty rows must not be saved.

---

# Fabric Price Calculation Rules

## Row Total Rules

- Each row must automatically calculate its own Row Total.
- Row Total field must be read-only.
- User must not manually edit Row Total.
- Row Total must dynamically update whenever:
  - Weight changes
  - Price Per Kg changes

---

## Total Price Rules

- Total Price must automatically calculate by adding all Row Totals.
- Formula:

```
Total Price = Sum of All Row Totals
```

- Total Price field must be read-only.
- User must not manually edit Total Price.
- Total Price must dynamically update whenever any row value changes.

---

# Fabric Submission Rules

- Fabric purchase data must be stored only after clicking Submit button.
- After successful save:
  - success message must be displayed
  - user must be redirected to Dashboard
- Failed save operation must display proper error message.

---

# Accessories Module Rules

## Accessory Selection Rules

- User must first select Accessory Name from dropdown.
- Different accessory types must dynamically display different input fields.
- Accessory Name field is mandatory.

---

# Cone Accessory Rules

## Cone Fields

Cone accessory must contain:

- Colour Name
- Colour Code
- Unit
- Price Per Unit

---

## Colour Name Rules

- Colour Name is mandatory.
- Colour Name cannot be empty.

---

## Colour Code Rules

- Colour Code is mandatory.
- Colour Code cannot be empty.

---

## Unit Rules

- Unit must be numeric.
- Unit cannot be negative or zero.
- Unit field is mandatory.

---

## Price Per Unit Rules

- Price Per Unit must be numeric.
- Price cannot be negative or zero.
- Currency symbol ₹ must always be displayed.
- User must not be allowed to edit or remove currency symbol.

---

# Dynamic Row Rules — Cone Module

- User can dynamically add multiple rows using Add button.
- Newly added rows must contain:
  - Colour Name
  - Colour Code
  - Unit
  - Price Per Unit
- Multiple rows belong to same cone purchase entry.
- Each row must independently calculate its own Row Total using:
  - Unit × Price Per Unit
- A Total Price must be displayed at the bottom.
- Total Price must be calculated by adding all Row Totals together.
- Empty rows must not be saved.

---

# Cone Total Price Rules

- Row Total must automatically calculate for each row.
- Formula:

```
Row Total = Unit × Price Per Unit
```

- Total Price must automatically calculate by adding all Row Totals.
- Formula:

```
Total Price = Sum of All Row Totals
```

- Row Total fields must be read-only.
- Total Price field must be read-only.
- User must not manually edit Row Total or Total Price.
- Total Price must dynamically update whenever any row value changes.

---

# Size Pattern Rules

## Required Fields

Size Pattern accessory must contain:

- Brand Name
- Style Number
- Style Name
- Price

---

## Brand Name Rules

- Brand Name is mandatory.
- Brand Name cannot be empty.

---

## Style Number Rules

- Style Number must be numeric.
- Style Number is mandatory.

---

## Style Name Rules

- Style Name is mandatory.
- Style Name cannot be empty.

---

## Price Rules

- Price must be numeric.
- Price cannot be negative or zero.
- Currency symbol ₹ must always be displayed.

- User must not be allowed to edit or remove currency symbol.

---

# Others Accessory Rules

## Required Fields

Others accessory must contain:

- Name
- Unit / Kg / Gross
- Price

---

## Name Rules

- Name field is mandatory.
- Name cannot be empty.

---

## Unit / Kg / Gross Rules

- Value must be numeric.
- Value cannot be negative or zero.
- Field is mandatory.

---

## Price Rules

- Price must be numeric.
- Price cannot be negative or zero.
- Currency symbol ₹ must always be displayed.
- User must not be allowed to edit or remove currency symbol.

---

# Submission Rules — Accessories

- Accessory purchase data must be stored only after clicking Submit button.
- After successful save:
    - success message must be displayed
    - user must be redirected to Dashboard

- Failed save operation must display proper error message.

---

# Database Rules

- All IDs must be auto-generated.
- Primary keys must be unique.
- Foreign key relationships must be maintained properly.
- Null values must not be allowed in mandatory fields.
- Data types must strictly follow database blueprint.

---

# API Rules

- Application must use REST API architecture.
- GET APIs must only retrieve data.
- POST APIs must create new data.
- PUT APIs must update existing data.
- DELETE APIs must remove data.

---

# UI Rules

- Application must be mobile responsive.
- Forms must be user-friendly and simple.
- Validation messages must be clearly visible.
- Required fields must be properly indicated.
- Submit buttons must remain disabled if mandatory fields are empty.
- Dynamic rows must visually align properly.
- Read-only calculated fields must be visually distinguishable.

---

# Error Handling Rules

- Invalid input must display proper validation message.
- System errors must display user-friendly messages.
- Application must prevent saving incomplete or invalid data.
- Empty dynamic rows must be ignored or removed automatically.

---

# Future Scope Rules

The architecture must support future implementation of:

- Payment Bills Module
- Expense Table View
- Search and Filters
- Reports
- PDF Export
- Analytics Dashboard
- Edit/Delete Operations
- Supplier Management
- Inventory Management
- Monthly Expense Tracking

# 9. FOLDER STRUCTURE REQUIREMENT

## Project Architecture

The application must follow:

- Layered Architecture
- MVC Pattern
- REST API Architecture
- Clean Code Structure
- Separation of Concerns Principle

The backend and frontend must be organized properly for scalability, maintainability, and future feature expansion.

---

# Recommended Project Structure

```
project-root/
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/fabricexpensemanagement/
│   │   │
│   │   │   ├── controller/
│   │   │   │
│   │   │   │   ├── AuthController.java
│   │   │   │   ├── DashboardController.java
│   │   │   │   ├── FabricPurchaseController.java
│   │   │   │   ├── AccessoryPurchaseController.java
```

```
│   │   │   │   ├── ConeController.java
│   │   │   │   ├── SizePatternController.java
│   │   │   │   └── OthersController.java
│   │   │   │
│   │   │   ├── service/
│   │   │   │
│   │   │   │   ├── AuthService.java
│   │   │   │   ├── DashboardService.java
│   │   │   │   ├── FabricPurchaseService.java
│   │   │   │   ├── AccessoryPurchaseService.java
│   │   │   │   ├── ConeService.java
│   │   │   │   ├── SizePatternService.java
│   │   │   │   └── OthersService.java
│   │   │   │
│   │   │   ├── repository/
│   │   │   │
│   │   │   │   ├── UserRepository.java
│   │   │   │   ├── FabricRepository.java
│   │   │   │   ├── AccessoryRepository.java
│   │   │   │   ├── ConeRepository.java
│   │   │   │   ├── SizePatternRepository.java
│   │   │   │   └── OthersRepository.java
│   │   │   │
│   │   │   ├── entity/
│   │   │   │
│   │   │   │   ├── User.java
│   │   │   │   ├── Fabric.java
│   │   │   │   ├── Accessory.java
│   │   │   │   ├── Cone.java
│   │   │   │   ├── SizePattern.java
│   │   │   │   └── Others.java
│   │   │   │
│   │   │   ├── dto/
│   │   │   │
│   │   │   │   ├── LoginRequestDTO.java
│   │   │   │   ├── RegisterRequestDTO.java
│   │   │   │   ├── FabricRequestDTO.java
│   │   │   │   ├── FabricResponseDTO.java
│   │   │   │   ├── AccessoryRequestDTO.java
│   │   │   │   └── AccessoryResponseDTO.java
│   │   │   │
│   │   │   ├── config/
│   │   │   │
│   │   │   │   ├── SecurityConfig.java
│   │   │   │   ├── SwaggerConfig.java
│   │   │   │   └── AppConfig.java
│   │   │   │
│   │   │   ├── exception/
│   │   │   │
│   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   ├── ResourceNotFoundException.java
│   │   │   │   ├── ValidationException.java
│   │   │   │   └── AuthenticationException.java
│   │   │   │
│   │   │   ├── util/
│   │   │   │
│   │   │   │   ├── PriceCalculatorUtil.java
│   │   │   │   ├── ValidationUtil.java
│   │   │   │   └── ResponseUtil.java
│   │   │   │
│   │   │   ├── constant/
```

```
│   │   │   │
│   │   │   │       ├── AppConstants.java
│   │   │   │       └── ApiConstants.java
│   │   │   │
│   │   │   └── FabricExpenseManagementApplication.java
│   │   │
│   │   ├── resources/
│   │   │
│   │   │   ├── application.properties
│   │   │   ├── static/
│   │   │   ├── templates/
│   │   │   └── data.sql
│   │   │
│   │   └── webapp/
│   │       │
│   │       ├── css/
│   │       ├── js/
│   │       ├── images/
│   │       └── pages/
│   │           ├── login.html
│   │           ├── register.html
│   │           ├── dashboard.html
│   │           ├── fabric-purchase.html
│   │           └── accessory-purchase.html
│   │
├── pom.xml
├── README.md
└── .gitignore
```

---

# Folder Responsibilities

---

# controller/

Responsible for:

- Handling API requests
- Receiving frontend requests
- Returning API responses

Controllers should:

- Contain only request handling logic
- Not contain business logic
- Call service layer methods

---

# service/

Responsible for:

- Business logic
- Calculations
- Data processing
- Validation coordination

Services should:

- Process application logic
- Interact with repositories
- Return processed results

---

# repository/

Responsible for:

- Database operations
- CRUD operations
- Query execution

Repositories should:

- Use Spring Data JPA
- Extend JpaRepository
- Contain database query methods

---

# entity/

Responsible for:

- Database table mapping
- JPA entity definitions

Entities should:

- Represent database tables
- Use proper annotations
- Maintain relationships

---

# dto/

Responsible for:

- Request body objects
- Response body objects
- API data transfer

DTOs should:

- Prevent direct entity exposure
- Improve security
- Separate API structure from database structure

---

# config/

Responsible for:

- Application configurations
- Security setup
- Swagger configuration
- Bean configurations

---

# exception/

Responsible for:

- Global exception handling
- Custom exception classes
- API error responses

---

# util/

Responsible for:

- Helper methods
- Common reusable logic
- Calculation utilities

---

# constant/

Responsible for:

- Static constant values
- API constants
- Application-wide constants

---

# resources/

Responsible for:

- Configuration files
- Static resources
- SQL scripts
- Application properties

---

# webapp/

Responsible for:

- Frontend files
- HTML pages
- CSS styles
- JavaScript files
- Images

---

# Frontend Folder Structure

```
webapp/
│
├── css/
│   ├── login.css
│   ├── dashboard.css
│   ├── fabric.css
│   └── accessory.css
│
├── js/
│   ├── login.js
│   ├── dashboard.js
│   ├── fabric.js
│   └── accessory.js
│
├── images/
│
└── pages/
```

```
├── login.html
├── register.html
├── dashboard.html
├── fabric-purchase.html
└── accessory-purchase.html
```

---

# Architecture Rules

- Controllers must not directly access repositories.
- Business logic must only exist inside service layer.
- Database operations must only happen through repositories.
- Entities must only represent database structure.
- DTOs must be used for API communication.
- Exception handling must be centralized.
- Utility functions must be reusable.
- Folder structure must remain modular and scalable.

---

# Naming Convention Rules

## Class Naming

- Use PascalCase for class names.

Example:

```
FabricPurchaseController
AccessoryService
UserRepository
```

---

## Method Naming

- Use camelCase for method names.

Example:

```
saveFabricPurchase()
getAllAccessories()
calculateTotalPrice()
```

---

## Variable Naming

- Use meaningful variable names.

Example:

```
totalPrice
fabricName
pricePerKg
```

---

# Future Scalability Requirement

The folder structure must support future modules including:

- Payment Bills
- Reports
- Expense Tables
- Analytics Dashboard
- Supplier Management
- Inventory Management
- Export Features

without major restructuring of the application architecture.

# 10. FUTURE FEATURES

## Future Scope Overview

The application architecture must be designed in a scalable and modular manner so that additional business modules and advanced features can be added in future versions without requiring major restructuring of the existing system.

The current version of the application mainly focuses on entering and storing purchasing expense data. Future versions will extend the application into a complete expense tracking and payment management system.

---

# 1. Payment Bills Module

## Purpose

The Payment Bills module will be used to manage and track payment-related details for all purchasing expenses.

This module will help the user:

- Track paid amounts
- Track pending amounts

- Maintain payment history
- Generate expense reports with payment information

---

## Planned Features

### Payment Entry

User will be able to:

- Add payment details for purchases
- Enter paid amount
- Enter payment date
- Enter payment method
- Track remaining balance

---

## Payment Status Tracking

The application should support:

- Fully Paid
- Partially Paid
- Pending Payment

statuses for each purchase record.

---

## Linked Purchase Details

Each payment record should be linked with:

- Fabric purchase details
- Accessory purchase details

so that complete expense history can be maintained.

---

# 2. Expense History Table

## Purpose

The application will include a complete expense table screen where the user can view all stored purchasing and payment records.

---

## Planned Features

The expense table should display:

- Fabric Name
- Fabric Type
- Colour
- GSM
- Weight
- Rib
- Price Per Kg
- Total Price
- Accessory Details
- Payment Status
- Paid Amount
- Pending Amount
- Purchase Date

---

## Table Features

The expense table should support:

- Search functionality
- Filter functionality
- Sorting
- Pagination
- Date-wise filtering
- Expense category filtering

---

# 3. PDF Report Generation

## Purpose

The application must support generating expense reports in PDF format.

This feature will allow the user to:

- Download expense history

- Share expense reports
- Maintain offline records
- Print reports if needed

---

# Planned PDF Content

The generated PDF should contain:

- Purchase details
- Fabric details
- Accessory details
- Payment details
- Total expense summary
- Pending payment summary
- Date-wise expense data

---

# PDF Data Fields

The PDF report may include:

- Fabric Name
- Fabric Type
- Colour
- GSM
- Weight
- Rib
- Price Per Kg
- Accessory Name
- Accessory Details
- Total Price
- Paid Amount
- Pending Amount
- Payment Status
- Purchase Date
- Payment Date

---

# PDF Export Features

The application should support:

- Download PDF
- Print PDF

- Generate monthly reports
- Generate date-range reports

---

# 4. Expense Analytics Dashboard

## Purpose

The application may later include graphical expense analytics.

---

## Planned Analytics

The dashboard may display:

- Monthly expense summary
- Fabric expense analysis
- Accessory expense analysis
- Pending payment analysis
- Expense trends

---

# 5. Search and Filter Module

## Planned Search Features

User should be able to search by:

- Fabric Name
- Fabric Type
- Colour
- Accessory Name
- Brand Name
- Style Number

---

## Planned Filter Features

User should be able to filter by:

- Date
- Expense category

- Payment status
- Price range
- Fabric type

---

# 6. Edit and Delete Features

## Planned Features

User should be able to:

- Edit purchase entries
- Delete incorrect entries
- Update payment records

---

# 7. Supplier Management Module

## Purpose

Future versions may include supplier management.

---

## Planned Features

The user may later:

- Store supplier details
- Link purchases with suppliers
- Track supplier-wise expenses

---

# 8. Inventory Management Module

## Purpose

The application may later expand into inventory management.

---

## Planned Features

Possible future features:

- Fabric stock tracking
- Accessory stock tracking
- Low stock alerts
- Stock usage history

---

# 9. Date and Time Tracking

## Planned Features

The application should later support:

- Purchase timestamps
- Payment timestamps
- Monthly expense history
- Daily expense summaries

---

# 10. Advanced Reporting

## Planned Features

Future reports may include:

- Monthly expense report
- Fabric-wise expense report
- Accessory-wise expense report
- Pending payment report
- Payment summary report

---

# 11. Mobile Optimization Enhancements

## Planned Improvements

Future versions may improve:

- Mobile responsiveness

- UI performance
- Faster loading
- Better user experience

---

# 12. Security Enhancements

## Planned Features

Future versions may include:

- JWT Authentication
- Session management
- Password reset functionality
- Better security configuration

---

# Future Architecture Requirement

The application architecture must remain:

- modular
- scalable
- maintainable

So future modules can be added without rewriting the existing application structure.

The backend, database structure, APIs, and frontend design should all support future expansion of:

- payment management
- reporting
- analytics
- PDF generation
- inventory tracking
- expense monitoring systems.

# Project logo:


Creators
Studio